INTRODUCTION TO PROGRAMMING • DIGITAL NOTES

UNIT III

Arrays and Strings

Indexing • Memory model • 1-D arrays • 2-D arrays • Strings & string functions

▶ What you will learn in this unit

- Why we need arrays
- Declaration & initialization
- Array indexing
- Memory model of arrays
- Programs on integer arrays
- Searching & sorting basics
- Two-dimensional arrays
- Matrix programs
- Introduction to strings
- String input / output
- String library functions
- Common mistakes & questions

3.1 Arrays: Concept, Declaration and Indexing

◆ Definition

An **array** is a collection of **same-type** elements stored in **continuous (adjacent) memory locations** and accessed with one name plus an **index**.

Without arrays, storing marks of 50 students needs 50 separate variables. With an array: `int marks[50];` — one name, 50 boxes, and a loop can process them all.

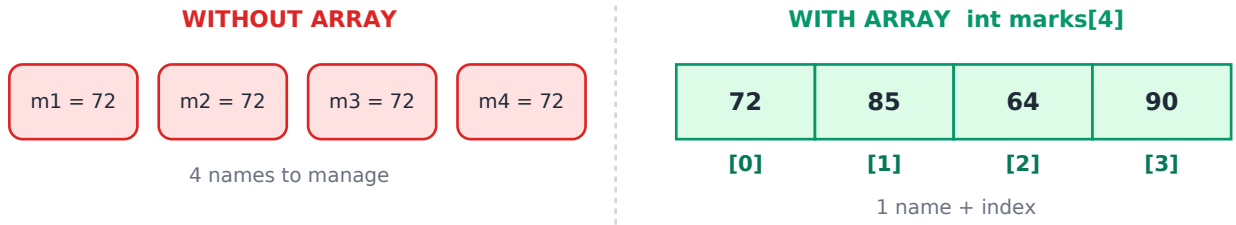


Fig 3.1 — Separate variables vs. one array

Task	Syntax	Example
Declaration	`type name[size];`	`int a[5];` (size must be a positive integer)
Initialization (full)	`type name[n] = {v1, v2, ...};`	`int a[5] = {10, 20, 30, 40, 50};`
Size from values	`type name[] = {...};`	`int a[] = {1, 2, 3};` → size 3
Partial	remaining become 0	`int a[5] = {1, 2};` → 1 2 0 0 0
All zeros	`= {0};`	`int a[100] = {0};`
Access / change	`name[index]`	`a[2] = 99; x = a[0];`

★ Exam Focus

Indexing starts at 0. An array of size **n** has valid indexes **0 to n−1**. The last element of `a[5]` is `a[4]`. C does **not** check bounds — `a[5]` or `a[-1]` silently reads/writes other memory (undefined behaviour).

Memory model of a 1-D array

Elements sit one after another. For `int a[5]` (4 bytes per int) starting at address 1000:

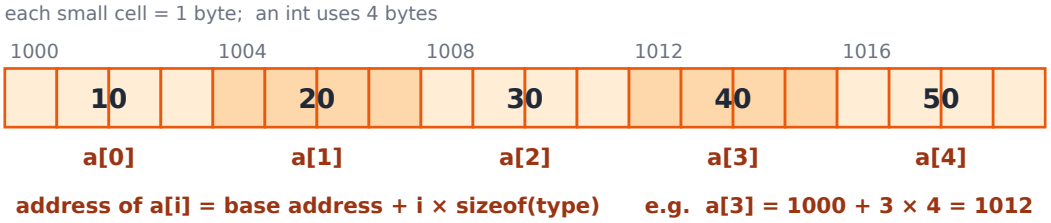


Fig 3.2 — Continuous memory layout of int a[5] (base address 1000)

✔ Remember
The array **name** itself is the **address of the first element** (`a == &a[0]`). This is why arrays and pointers are so closely related (Unit IV). Array size in bytes = `n × sizeof(type)`; number of elements = `sizeof(a) / sizeof(a[0])`.

3.2 Programs with Arrays of Integers

Traversing an array means visiting every element with a `for` loop from index **0** to **n−1**.

Read, sum and average

```c
int a[5], i, sum = 0;
printf("Enter 5 numbers: ");
for (i = 0; i < 5; i++) {          // reading: note the & before a[i]
    scanf("%d", &a[i]);
    sum += a[i];
}
printf("Sum = %d, Average = %.2f", sum, sum / 5.0);
```

OUTPUT

```
Enter 5 numbers:
10 20 30 40 50
Sum = 150, Average = 30.00
```

Largest and smallest

```c
int a[] = {42, 7, 56, 19, 33}, i, max, min;
max = min = a[0];                    // start with the first element
for (i = 1; i < 5; i++) {
    if (a[i] > max) max = a[i];
    if (a[i] < min) min = a[i];
}
printf("Max=%d Min=%d", max, min);
```

OUTPUT

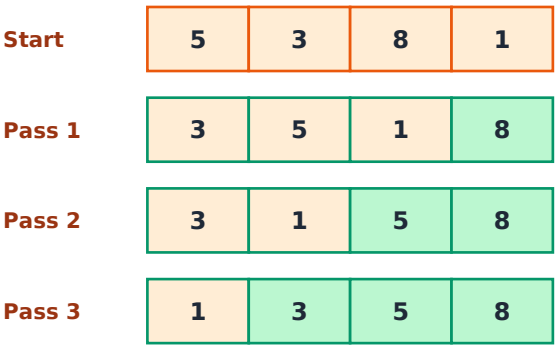
```
Max=56 Min=7
```

Linear search and reverse

```c
int key = 19, pos = -1;
for (i = 0; i < 5; i++)
    if (a[i] == key) { pos = i; break; }          // stop at first match
if (pos != -1) printf("Found at index %d", pos);
else           printf("Not found");
for (i = 4; i >= 0; i--) printf("%d ", a[i]);     // reverse printing
```

OUTPUT

```
Found at index 3
33 19 56 7 42
```



compare neighbours, swap if left > right

green = in final pla
biggest value "bubble" to the end each pass

BUBBLE SORT (ascending)

```c
for (i = 0; i < n-1; i++)
  for (j = 0; j < n-1-i; j++)
    if (a[j] > a[j+1]) {
      t = a[j];
      a[j] = a[j+1];   // swap
      a[j+1] = t;
    }
```

Fig 3.3 — Bubble sort on {5, 3, 8, 1}

Step	low	high	mid	a[mid]	Decision (key = 50)
1	0	6	3	40	40 < 50 → search right: low = mid+1
2	4	6	5	60	60 > 50 → search left: high = mid−1
3	4	4	4	50	**Found at index 4**

Binary search (trace table above)

```c
int a[] = {10, 20, 30, 40, 50, 60, 70};     // MUST be sorted
int lo = 0, hi = 6, mid, key = 50, pos = -1;
while (lo <= hi) {
    mid = (lo + hi) / 2;
    if      (a[mid] == key) { pos = mid; break; }
    else if (a[mid] <  key) lo = mid + 1;       // go right
    else                    hi = mid - 1;       // go left
}
```

OUTPUT

```
pos = 4
```

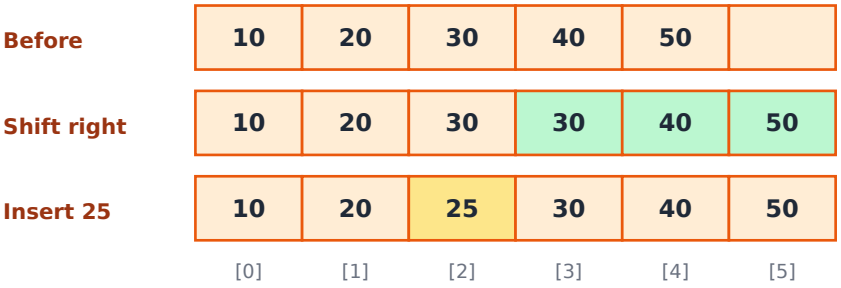


Fig 3.4 — Inserting 25 at position 2 (elements shift right, last element first)

INSERT / DELETE

```c
for (i = n-1; i >= pos; i--)
    a[i+1] = a[i]; // room
a[pos] = val;  n++;

// DELETE at pos:
for (i = pos; i < n-1; i++)
    a[i] = a[i+1]; // gap
n--;
```

Count even/odd and merge arrays

```c
int a[] = {42, 7, 56, 19, 33}, i, ev = 0, od = 0;
for (i = 0; i < 5; i++)
    if (a[i] % 2 == 0) ev++; else od++;          // even / odd test
printf("Even=%d Odd=%d\n", ev, od);
int A[] = {1, 3, 5}, B[] = {2, 4}, C[5];         // merge two arrays
for (i = 0; i < 3; i++) C[i] = A[i];
for (i = 0; i < 2; i++) C[3 + i] = B[i];         // continue after A
```

OUTPUT

```
Even=2 Odd=3
C = 1 3 5 2 4
```

Advantages of arrays	Limitations of arrays
Many values under one name; easy loop processing	Fixed size — cannot grow or shrink at run time
Fast direct access by index — O(1)	Insertion / deletion are slow (elements must shift)
Contiguous memory = cache friendly	All elements must be of the same type
Base for strings, matrices, stacks, queues	No bounds checking by the compiler

● Real-life Use

Marks lists, temperature logs, game boards (tic-tac-toe = 3×3 array), image pixels (2-D), spreadsheets, and storing text as character arrays.

Algorithm	Best	Worst	Needs sorted data?
Linear search	O(1)	O(n)	No
Binary search	O(1)	O(log n)	**Yes**
Bubble sort	O(n)	$O(n^2)$	—

3.3 Two-Dimensional Arrays

◆ Definition

A **2-D array** is an "array of arrays" — a table of **rows** and **columns**. Declared as `type name[ROWS][COLS];` and accessed as `name[row][col]`.

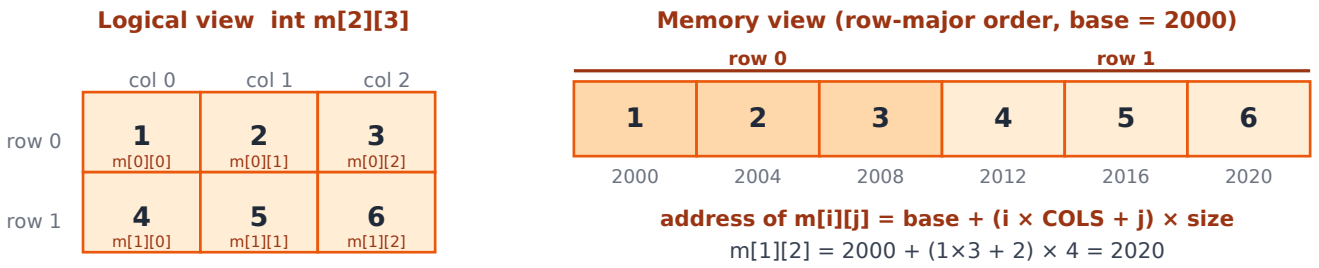


Fig 3.5 — A 2-D array in logical and memory view

Reading and printing a matrix

```c
int m[2][3] = {{1, 2, 3}, {4, 5, 6}};     // row by row
int i, j;
for (i = 0; i < 2; i++) {                  // outer loop = rows
    for (j = 0; j < 3; j++)                // inner loop = columns
        printf("%d ", m[i][j]);
    printf("\n");
}
```

OUTPUT

```
1 2 3
4 5 6
```

Matrix operations — all use nested loops:

Matrix addition and transpose

```c
for (i = 0; i < 2; i++)
    for (j = 0; j < 2; j++)
        C[i][j] = A[i][j] + B[i][j];      // ADDITION

for (i = 0; i < 2; i++)
    for (j = 0; j < 3; j++)
        T[j][i] = m[i][j];                // TRANSPOSE
```

OUTPUT

```
C =  6  8
    10 12

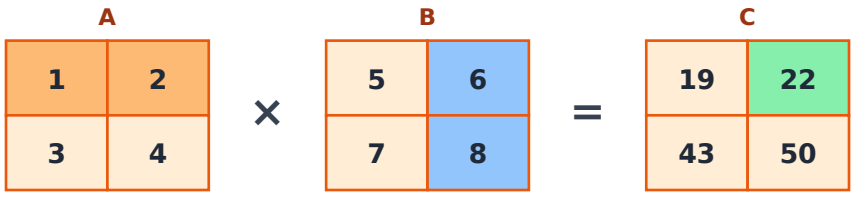
T =  1  4
     2  5
     3  6
```

Matrix multiplication C = A x B

```c
for (i = 0; i < 2; i++)
    for (j = 0; j < 2; j++) {
        C[i][j] = 0;
        for (k = 0; k < 2; k++)
            C[i][j] += A[i][k] * B[k][j];     // row of A  x  column of B
    }
```

OUTPUT

```
C = 19 22
    43 50
```



C[0][1] = (row 0 of A) · (col 1 of B)

= 1×6 + 2×8 = 6 + 16 = 22

orange = row of A, blue = column of B, green = resulting element

Fig 3.6 — How one element of the product matrix is computed

Row sums and diagonal sum of a 3x3 matrix

```c
int m[3][3] = {{1,2,3}, {4,5,6}, {7,8,9}}, i, j, rs, dsum = 0;
for (i = 0; i < 3; i++) {
    rs = 0;
    for (j = 0; j < 3; j++) rs += m[i][j];       // sum of row i
    printf("Row %d sum = %d\n", i, rs);
    dsum += m[i][i];                             // main diagonal
}
printf("Diagonal sum = %d", dsum);
```

OUTPUT

```
Row 0 sum = 6
Row 1 sum = 15
Row 2 sum = 24
Diagonal sum = 15
```

Operation	Rule	Result order	Loops
Addition / subtraction	same order; C[i][j] = A[i][j] ± B[i][j]	m × n	2
Transpose	T[j][i] = M[i][j]	n × m	2
Multiplication	cols of A = rows of B	m × p	3
3-D array	`int c[2][3][4];` = 2 tables of 3 rows × 4 columns	—	3

✔ Remember
Multiplication rule: A(m×n) × B(n×p) = C(m×p) — columns of A must equal rows of B. Example: C[0][0] = 1×5 + 2×7 = **19**. Addition needs both matrices of the **same order**.

3.4 Introduction to Strings

◆ Definition
A **string** is a one-dimensional **array of characters** that ends with the **null character** `\0` (ASCII 0). The `\0` tells functions where the string stops.

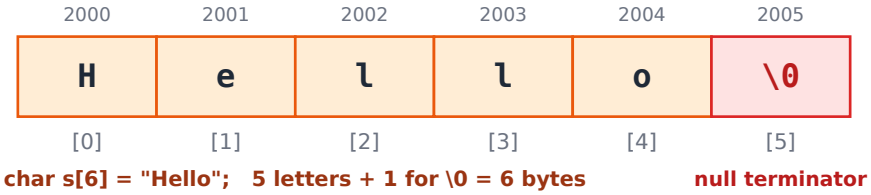


Fig 3.7 — A string in memory (each char = 1 byte)

Way	Code	Note
Literal	`char s[] = "Hi";`	size = 3 (H, i, \0), compiler adds \0
Fixed size	`char s[20] = "Hi";`	rest of array filled with \0
Char list	`char s[] = {'H','i','\0'};`	you must add \0 yourself
Pointer	`char *s = "Hi";`	string literal — cannot be modified

⚠ Common Mistake
"A" is a string (2 bytes: A and \0) but **'A'** is a single char (1 byte). Never compare strings with `==` or copy with `=`; use `strcmp()` and `strcpy()`.

Characters and ASCII values

Every character is stored as a number (ASCII code), so characters can be used in arithmetic:

Character	'0' … '9'	'A' … 'Z'	'a' … 'z'	space	'\n'	'\0'
ASCII code	48 … 57	65 … 90	97 … 122	32	10	0

Character arithmetic

```
char c = '7';
int digit = c - '0';        // 55 - 48 = 7  (char -> number)
char up = 'a' - 32;         // 97 - 32 = 65  -> 'A'      (or toupper('a'))
char nx = 'A' + 2;          // 65 + 2  = 67  -> 'C'
int diff = 'z' - 'a';       // 25
```

OUTPUT

```
digit = 7
up    = A
nx    = C
diff  = 25
```

String input and output

Function	Purpose	Remark
`scanf("%s", s)`	Reads one word	Stops at space; **no & needed**; risk of overflow
`fgets(s, size, stdin)`	Reads a full line	Safe; keeps the newline `\n`
`gets(s)`	Reads a line	**Unsafe, removed in C11** — avoid
`printf("%s", s)` / `puts(s)`	Prints string	`puts` adds a newline automatically
`getchar()` / `putchar(c)`	One character in / out	Used in character-by-character loops

Reading a line with spaces

```c
char name[30];
printf("Enter your full name: ");
fgets(name, 30, stdin);      // %s would stop at space
printf("Hello, %s", name);
```

OUTPUT

```
Enter your full name:
Asha Rao
Hello, Asha Rao
```

String library functions (#include <string.h>)

Function	What it does	Example	Result
`strlen(s)`	Length (without \0)	`strlen("Hello")`	5
`strcpy(d, s)`	Copy s into d	`strcpy(d, "Hi")`	d = "Hi"
`strcat(d, s)`	Append s to d	`strcat("Good", "Day")`	"GoodDay"
`strcmp(a, b)`	Compare: 0 if equal, <0 if a<b, >0 if a>b	`strcmp("apple","banana")`	negative
`strncpy / strncat`	Same, but limited to n chars (safer)	`strncpy(d, s, 3)`	copies 3 chars
`strchr(s, c) / strstr(s, t)`	Find a character / substring	`strstr("hello","ll")`	pointer to "llo"

ctype.h	Use	ctype.h	Use
`toupper(c)`	'a' → 'A'	`isalpha(c)`	letter?
`tolower(c)`	'A' → 'a'	`isdigit(c)`	digit 0-9?

Count vowels

```c
char s[] = "education";  int i, v = 0, ch;
for (i = 0; s[i] != '\0'; i++) {       // until null char
    ch = tolower(s[i]);
    if (ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u') v++;
}
printf("Vowels = %d, Consonants = %d", v, i - v);
```

OUTPUT

```
Vowels = 5,
Consonants = 4
```

Palindrome check

```c
char s[] = "madam";
int i = 0, j = strlen(s) - 1, ok = 1;
while (i < j) {                        // compare first with last, move inwards
    if (s[i] != s[j]) { ok = 0; break; }
    i++;  j--;
}
printf(ok ? "Palindrome" : "Not a palindrome");
```

OUTPUT

```
Palindrome
```

Length without strlen & array of strings

```c
char s[] = "Programming";  int len = 0;
while (s[len] != '\0') len++;                // own strlen
printf("Length = %d\n", len);
char days[3][10] = {"Mon", "Tue", "Wed"};   // array of strings
printf("%s", days[1]);
```

OUTPUT

```
Length = 11
Tue
```

string.h in action

```c
char a[20] = "Good", b[20];
strcpy(b, "Day");                // b = "Day"
strcat(a, " ");  strcat(a, b);  // a = "Good Day"
printf("%s (%d chars)\n", a, strlen(a));
printf("%d\n", strcmp("abc", "abc"));       // 0  -> equal
```

OUTPUT

```
Good Day (8 chars)
0
```

Sorting names alphabetically

```c
char n[3][10] = {"Ravi", "Amit", "Kiran"}, t[10];
for (i = 0; i < 2; i++)                          // bubble sort on strings
    for (j = 0; j < 2 - i; j++)
        if (strcmp(n[j], n[j+1]) > 0) {          // out of alphabetical order?
            strcpy(t, n[j]); strcpy(n[j], n[j+1]); strcpy(n[j+1], t);
        }
```

OUTPUT

```
Amit
Kiran
Ravi
```

Word count and uppercase

```c
char s[] = "hello world";  int i, words = 1, len = strlen(s);
for (i = 0; i < len; i++)
    if (s[i] == ' ') words++;        // a space = new word
for (i = 0; i < len; i++) s[i] = toupper(s[i]);
printf("%s has %d words", s, words);
```

OUTPUT

```
HELLO WORLD has
2 words
```

★ Exam Focus

Common errors: (1) forgetting space for `\0` → `char s[5] = "Hello"` overflows; (2) using `==` for strings; (3) accessing `a[n]`; (4) not using `&` with `scanf("%d", &a[i])` (but strings with `%s` need no &); (5) arrays cannot be assigned or compared directly: `a = b` is illegal.

3.5 More Solved Programs (Exam Favourites)

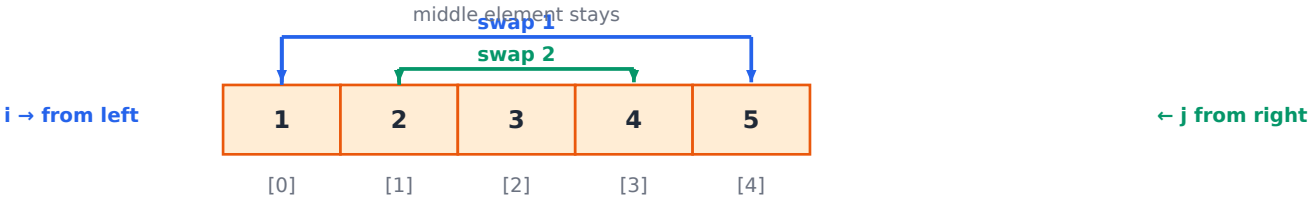


Fig 3.8 — Reversing an array in place: swap a[i] with a[j] until i ≥ j

Reverse in place and second largest

```c
int a[] = {1, 2, 3, 4, 5}, i, j, t;
for (i = 0, j = 4; i < j; i++, j--) {      // two indexes
    t = a[i];  a[i] = a[j];  a[j] = t;
}
// Second largest of {42, 7, 56, 19, 33}
int b[] = {42, 7, 56, 19, 33}, first = -1, second = -1;
for (i = 0; i < 5; i++) {
    if (b[i] > first)                  { second = first; first = b[i]; }
    else if (b[i] > second && b[i] != first)  second = b[i];
}
```

OUTPUT

```
a = 5 4 3 2 1
first  = 56
second = 42
```

Character frequency (counting array trick)

```c
char s[] = "banana";  int count[26] = {0}, i;
for (i = 0; s[i]; i++) count[s[i] - 'a']++;     // 'a'->0 ... 'z'->25
for (i = 0; i < 26; i++)
    if (count[i]) printf("%c=%d ", 'a' + i, count[i]);
```

OUTPUT

```
a=3 b=1 n=2
```

Symmetric matrix check

```c
int m[3][3] = {{1,2,3}, {2,5,6}, {3,6,9}}, sym = 1;
for (i = 0; i < 3; i++)
    for (j = 0; j < 3; j++)
        if (m[i][j] != m[j][i]) sym = 0;          // symmetric: m == transpose
printf(sym ? "Symmetric" : "Not symmetric");
```

OUTPUT

```
Symmetric
```

Task	Loop pattern	Remember
Visit every element	`for (i = 0; i < n; i++)`	last index is n − 1
Compare with the first / start from second	`max = a[0]; for (i = 1; ...)`	initialize with a real element
Compare neighbours (sorting)	`for (j = 0; j < n-1-i; j++)`	a[j+1] must stay inside the array
Work from both ends	`for (i=0, j=n-1; i < j; i++, j--)`	stop when the indexes meet
Every cell of a matrix	nested: outer = rows, inner = columns	m[i][j] → i row, j column
Walk through a string	`for (i = 0; s[i] != '\0'; i++)`	no need to know the length

★ UNIT SUMMARY — QUICK REVISION

- **Array** = same-type elements in **contiguous memory**; index runs **0 … n−1**; no bounds checking.
- Address of `a[i]` = **base + i × size**; the array name is the address of the first element.
- Process arrays with `for` loops: sum, max/min, linear search O(n), binary search O(log n) on sorted data, bubble sort O(n^2).
- **2-D array** `m[r][c]` is stored **row-major**; address = base + (i×COLS + j)×size; nested loops for matrix add, transpose, multiply.
- **String** = char array ending with **\0**; size needed = characters + 1. Use **fgets** for lines, **%s** for one word.
- `strlen, strcpy, strcat, strcmp` live in `<string.h>`; `strcmp` returns **0 when equal**.

Q Exam-Oriented Questions

Short Answer (2 marks)

1. Define an array. Why are arrays needed?
2. What are the valid indexes of `int a[10]`? What happens if `a[10]` is accessed?
3. How is a 1-D array stored in memory? Give the address formula.
4. Write the declaration of a 3×4 integer matrix and initialize it.
5. What is the role of the null character in a string?
6. Differentiate between `scanf("%s")` and `fgets()`.
7. Why can't two strings be compared using `==`?
8. What are the values returned by `strcmp()`?
9. Differentiate between `'A'` and `"A"`.
10. What is the time complexity of linear and binary search?

Long Answer / Programs (5–10 marks)

1. Explain declaration, initialization and accessing of 1-D arrays with examples.
2. Write programs to (a) find the largest and smallest, (b) search an element, (c) reverse an array.
3. Write a program to sort an array using bubble sort and show the pass-wise trace.
4. Explain 2-D arrays with memory layout. Write a program for matrix addition and multiplication.
5. Write a program to find the transpose of a matrix and the sum of its diagonal elements.

6. Explain any six string-handling functions with examples.
7. Write programs to (a) find the length of a string without strlen, (b) check a palindrome, (c) count vowels.
8. Write a program to read a line of text and convert it to uppercase.